Gradient Descent, Momentum, and Hessian Eigenvalues

Purpose. These notes collect our discussion into one worked example showing why curvature controls the learning rate and why momentum can accelerate optimization in ill-conditioned valleys.

1. Quadratic model and curvature

Consider the two-dimensional loss

$$L(x,y) = 1/2\,(x^2 + 100\,y^2).$$

Its gradient is $\nabla L = (x, 100y)^T$ and its Hessian is $H = \mathrm{diag}(1,100)$. Therefore the Hessian eigenvalues are $\lambda_{min} = 1$ and $\lambda_{max} = 100$. The x direction is relatively flat, while the y direction is 100 times more curved.

The condition number is $\kappa = \lambda_{max}/\lambda_{min} = 100$. A large condition number produces a long, narrow valley: a step size that is safe in the steep direction can be frustratingly small in the flat direction.

2. Plain gradient descent

Gradient descent updates the parameters according to

$$\theta_{k+1} = \theta_k - \alpha\,\nabla L(\theta_k),$$

where α is the learning rate. Along an eigenvector with eigenvalue λ, the error is multiplied each step by $(1 - \alpha\lambda)$. Thus large eigenvalues impose the strongest stability restriction.

For this quadratic, using $\alpha = 0.019$ and starting at (1,1):

x: $1 \to 0.981 \to 0.9624 \to 0.9441 \to \ldots$
y: $1 \to -0.9 \to 0.81 \to -0.729 \to \ldots$

The y component repeatedly changes sign because $1 - 0.019(100) = -0.9$. This is the familiar zigzag across the narrow valley. Meanwhile x decreases slowly because $1 - 0.019(1) = 0.981$.

3. Momentum (heavy-ball method)

Momentum introduces a velocity variable:

$v_{k+1} = \beta v_k - \alpha \nabla L(\theta_k)$,
$\theta_{k+1} = \theta_k + v_{k+1}$.

Here β is the momentum coefficient. A larger β means more memory of previous updates, not simply a universally larger speed. When gradients point consistently in one direction, velocity can accumulate. When a component repeatedly reverses direction, past velocity can counteract part of that oscillation.

This is why α and β must be considered together. A large learning rate combined with excessive momentum can overshoot or become unstable.

4. Curvature-matched parameters

For a positive-definite quadratic with eigenvalues between $\mu = \lambda_{min}$ and $L = \lambda_{max}$, the classical heavy-ball choices are

$\alpha^* = 4 / ((\sqrt{L} + \sqrt{\mu})^2)$,

$\beta^* = ((\sqrt{L} - \sqrt{\mu})/(\sqrt{L} + \sqrt{\mu}))^2$.

For $\mu = 1$ and $L = 100$, these give $\alpha^* = 0.033058 \approx 0.0331$ and $\beta^* = 0.669421 \approx 0.6694$.

These values are optimized for the asymptotic convergence rate of this ideal quadratic model. They do not guarantee that every coordinate moves monotonically; some early oscillation or overshoot can still occur.

5. Geometric picture

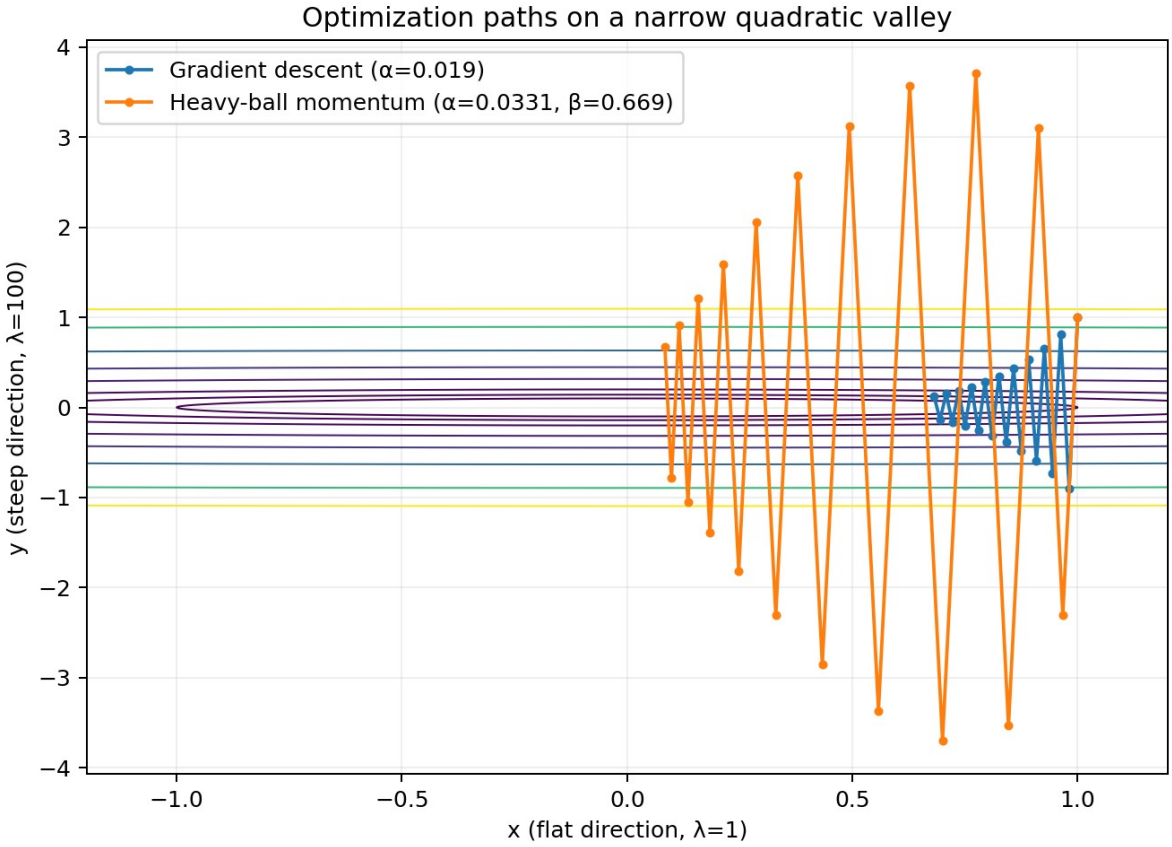


Figure 1. Gradient descent and heavy-ball momentum on $L(x,y)=\frac{1}{2}(x^2+100y^2)$. The contours reveal the narrow valley created by the unequal Hessian eigenvalues.

6. Why momentum can be much faster

For strongly convex quadratics, the convergence difficulty of ordinary gradient descent is governed by the condition number $\kappa = L/\mu$. Properly tuned momentum improves the dependence from roughly κ to roughly $\sqrt{\kappa}$. In this example, $\kappa = 100$ while $\sqrt{\kappa} = 10$, which captures the potential acceleration.

The intuition is: steep directions restrict the safe learning rate; flat directions then progress slowly. Momentum stores useful motion in directions where gradients remain consistent, while reducing the cost of repeated back-and-forth motion across the valley.

7. Connection to Adam

Adam extends this intuition by combining a momentum-like first-moment estimate with an adaptive second-moment estimate. In simplified form, it maintains an exponential average of gradients and an exponential average of squared gradients, then scales each parameter's update using those quantities. This gives different effective step sizes across coordinates, which can be helpful when curvature and gradient scales differ substantially.

In deep neural networks the loss is not a fixed quadratic and its Hessian changes throughout training, so the exact heavy-ball formulas are mainly a conceptual guide. In practice, learning rate and momentum are tuned empirically while monitoring convergence and stability.

Key takeaway

Eigenvalues describe curvature. The largest eigenvalue limits how aggressive the learning rate can be; the ratio $\lambda_{max}/\lambda_{min}$ measures how poorly conditioned the landscape is. Momentum adds memory, allowing optimization to accumulate motion along persistent directions and handle narrow valleys more efficiently when α and β are well matched.